

Department of Computer Science



Submitted in part fulfilment for the degree of BEng.

A Web-based Teaching Tool for Hoare Logic

Stephen Michael Ingham

Version 4.0, 2025-April

Supervisor: Simon David Foster

Acknowledgements

I would like to thank my academic supervisor, Dr. Simon David Foster, and academic colleague, Christian Pardillo Laursen, for their insights and technical advice in aiding the direction and development of this project.

Contents

Executive Summary	iv
Statement of Ethics	vi
0.1 Avoidance of Harm	vi
0.2 Informed Consent	vi
0.3 Data Protection	vi
1 Introduction	1
2 Literature Review	3
2.1 Overview	3
2.1.1 Brief Introduction to Hoare Logic	3
2.1.2 Industry and Teaching Applications of Hoare Logic	4
2.1.3 Context of Teaching Environment and Resulting Research Question	5
2.2 Systematic Review of Existing Tooling	5
2.2.1 Introduction	5
2.2.2 Existing Tooling	5
2.2.3 Tooling Review Outcome	7
3 Methodology	8
3.1 Elicitation of Requirements	8
3.1.1 User Survey	9
3.1.2 Stakeholder Requirements	9
3.1.3 Review Notes	10
3.1.4 Analysis of Survey Results	10
3.1.5 Approach to Reconcile Collected Data	11
3.2 Resulting Software Requirements	12
3.3 Intended Proof Form	14
3.4 Development Strategies	16
3.5 Chosen Implementation Approach	17
3.6 Software Decisions	18
3.7 Application Process Flow	20
3.8 Design of Tooling Software	21
3.9 Brief Overview of Prototype Evaluation Criteria	23
4 Implementation	24
4.1 Overview	24

Contents

4.2	Design	24
4.2.1	Frontend Interface	24
4.2.2	Proof Kernel using Microsoft Z3	25
4.3	Encountered Issues & Limitations	27
4.4	Implementation Summary	27
5	Evaluation and Testing	29
5.1	Testing	29
5.1.1	Automated Testing	29
6	Conclusion	32
A	Appendices	33
B	Link to Github Repository	34
C	Survey Data	35
D	Additional Figma Design Images	42
E	Additional Prototype Images	44
F	All Syntax Error Rules Table	48
G	Unit Test Results	50
H	Accessibility Test Results	53
H.1	Z3 WASM Load Time Tests	53
H.2	Adherence to WCAG 2.1 Guidelines	53

List of Figures

3.1	Application Process Diagram	20
3.2	High-fidelity Design of Landing Page of Tool	22
3.3	High-fidelity Design of Reference Guide Page	22
D.1	After Pressing 'Check Proof Validity' Button	43
E.1	Landing Page (Maximised Sidebar)	45
E.2	Reference Guide (Top Half)	45
E.3	Valid Proof	46
E.4	Invalid Proof	46
E.5	Invalid Proof with Syntax Error	47
G.1	Skip Law Valid Proof	51
G.2	Skip Law Valid Proof	51
G.3	Summary of all Valid and Invalid Proofs	52

List of Tables

3.1	Functional Requirements Table	12
3.2	Non-Functional Requirements Table	13
3.3	Suffix for Proof Rule	15
3.4	SMT Solver Benefits and Drawbacks	19
4.1	Syntax Error Checks For Substitution Rule	25
5.1	Examples of Valid and Invalid Assignment Law Proofs . . .	30
5.2	Evaluation Against Success Criteria	31
F.1	Syntax Error Checks For Rule	49
H.1	WASM module load time test (seconds)	53

Executive Summary

Hoare Logic is a proof system used for determining the mathematical correctness of software through demonstration of adherence to its axioms, known more informally as the laws of Hoare Logic. The proof system is used in the systematic verification of software, largely within safety-critical domains such as avionics[1]. The system is used as the base for many prominent Program Verifiers and Automated Theorem Provers(ATP) such as Isabelle/HOL[2], however it faces the prominent issue alongside many Theoretical Computer Science(TCS) topics that it is notoriously hard to teach.

While a number of approaches can be taken in providing improved teaching delivery and consequently on a students ability to learn the tenants of the system, a growing base of studies evidence the value that teaching tools in formal methods as an aid in the teaching process can have on learning outcomes. However, in the area of tooling for Hoare Logic specifically, there is a shortfall in solutions which cater to the usability needs of undergraduate Computer Science students. Namely, teaching tooling typically suffers from being difficult to setup and providing a complex interface for working with among other issues.

This lack of suitable tooling provided the inspiration for this project, in which the aim was devised to develop a software prototype which directly addressed these issues and worked to overcome them in order to produce an effective aid for use by instructors at the University of York and beyond. The determination of lack of suitable tooling was motivated by a thorough review of teaching tooling in the area of formal methods at the outset of the project which not only established the need for the project but also informed some of the criteria of what could constitute an effective tool. Objectives were defined to break down the project into measurable and practical steps: Elicit suitable requirements for a software prototype; Determine a viable technical architecture upon such requirements; Produce a design mock up of the tool interface for stakeholder approval; Implement the prototype according to the design produced; Evaluate the developed tool's success against the project aims through the extent of its compliance with the projects success criteria.

The following methodology was adhered to in order to meet these object-

Executive Summary

ives:

- Conduct a systematic literature review of comparable teaching tools
- Conduct a user survey and gather requirements
- Consider design approaches and subsequently build the application
- Test and Evaluate success of the application

The Waterfall method[3] was adhered to as the software development methodology for the project. The rationale being as the requirements for the software once established from the systematic literature review and user survey would be set due to the time restrictions of the project making a more agile development approach impractical.

The project's systematic literature review uncovered the major themes in usability access in contemporary teaching tooling in formal methods being around difficulty in software setup, lack of instant feedback and the learning curves most of such tooling posed. The review overall highlighted there was a lack of a tool which solved all of the issues mentioned, thus as a result it was determined a software solution needed to be implemented. In order to aid the requirements of a solution, a user survey was needed to provide more tangible and direct input into the tooling to be developed. The outputs of the survey were also combined with input from the project stakeholders, Simon David Foster and Christian Pardillo Laursen, which allowed for the elicitation of Functional and Non-Functional requirements. These requirements were then considered in the context of the possible development strategies and the chosen implementation was to develop an accessible web-enabled frontend running a proof kernel in the browser using the Simple Modulo Transformer solver Microsoft Z3[4]. The software was thoroughly tested by means of unit tests and other automated testing tooling such as Google Lighthouse[5] to test the range of Hoare Logic proofs supported by the tooling and the range of syntax errors that could be reliably identified. More subjective criteria such as visual design aspects and general usability was evaluated in its own context. The tool met 9/13 of the essential criterion, in the case of unfulfilled requirements these were unfortunately due to the project time constraints however the requirements that were met were done so to a high standard.

The results of the evaluation evidenced the advancement of the state of the art in teaching tooling by producing an easy to access proof kernel through the medium of a web application with support for instant feedback through syntax error highlighting and quick validity conclusions on Hoare Triple proofs. Hence, the project has adhered to the derived requirements specification and thus has met its aims.

Statement of Ethics

0.1 Avoidance of Harm

Anonymized survey data from university students (18+) was collected via a web-based host for software requirements gathering. The study posed minimal physical and arguably very low mental harm due to the nature of the questions about learning formal logic and tooling. Foreseeable adverse applications of the project findings are unlikely, as the outcomes are intended to likely only benefit the teaching of formal logic in Computer Science education.

0.2 Informed Consent

Participants were required to read through and agree to the terms of the study as detailed in the informed consent sheet produced ahead of participation. This sheet was created during completion of the fast track application form to gain ethical approval for the study to commence for which it did. As a result, the constraints detailed in Article 3.2 of the University Code of Ethics were adhered to.

0.3 Data Protection

The university securely stored anonymized survey data, acting as the Data Controller under UK GDPR and the Data Protection Act (2018). Access was restricted to project investigators via a password-protected folder. Qualitative data was analyzed both individually (with pseudonyms) and in aggregate, while quantitative data was analyzed solely in aggregate to identify patterns. Anonymization was achieved through pseudonymisation of participant email addresses upon data receipt.

1 Introduction

Hoare Logic[6] is a type of mathematical proof logic, by which proof laws expressed in propositional logic are used to decompose and verify the mathematical correctness of a software program, i.e. proving a Hoare Triple.

Hoare Logic belongs to a group of techniques known as formal methods, a branch of software engineering ensuring the development of high-integrity software, such as software developed for safety-critical systems, e.g. inspiring formal verification approaches applicable to avionics control systems[7]. The principles of Hoare Logic are often used as a base logic for program verifiers of imperative code, such as in the verification properties of the Dafny language[8] and the deductive verification plugins of Frama-C such as Jessie[9].

As a result, formal methods teaching, specifically with the Hoare Logic Proof System, is a common and fundamental topic in the delivery of many Computer Science Undergraduate programmes when Theoretical Computer Science (TCS) topics are covered such as at the University of York[10].

However teaching formal methods itself is often not an easy task, in part due to the inherent complexity of such topics but also the abstractness of the topic from any practical use cases. Attempts to improve the teaching quality of courses on TCS topics, have been made with varying degrees of success, e.g. providing lectures from guest speakers in industry to speak to practical relevance of TCS topics. However, with far more assurance, software tooling as an aid in the teaching of TCS topics, including formal methods, has been shown to provide a great deal of value to students ability to learn during teaching delivery, as evidenced by the impact using teaching tooling such as Carnap[11] and Key-Hoare[12].

At the time of writing, within the standard undergraduate Computer Science curriculum in the second year Theory module named 'Theory-3' there is no software tooling utilised to aid the teaching of the Hoare Logic Proof System, specifically the construction of Hoare Logic proofs. This is due in large part because, amidst the value existing teaching tools offer as just mentioned, which do support Hoare Logic proof construction, no tool exists which is suitable in terms of providing ease of access and low configuration

in order to use that will be required by Undergraduate University students.

Therefore, the aim of this project is to enhance a students ability to learn how to construct Hoare Logic proofs by developing easy to access but nonetheless sufficient proof kernel software which allows for the verification of Hoare Logic proofs. The manner in which this aim will be achieved is by designing a tool which builds on the state of the art in proof assistant tooling within a university classroom, as will be detailed in the subsequent chapter.

In order to achieve this aim, the project was broken down into the following objectives:

- Elicit suitable requirements for a software prototype. This is performed by utilizing user survey data alongside literature on comparable tooling, identifying the features of the tool which concern its pedagogical effectiveness
- Determine a viable technical architecture upon such requirements
- Produce a design mock up of the tool interface for stakeholder approval,
- Implement the prototype according to the design produced
- Evaluate the developed tool's success against the project aims through the extent of its compliance with the projects success criteria

2 Literature Review

2.1 Overview

In this chapter we briefly introduce the Hoare Logic proof system formally and some motivating examples of its practical usage, proceeding into an overview of how it is currently taught within the University of York's Computer Science Curricula.

We will then describe the resulting research question proposed which provided the motivation for this project at large, followed by the systematic review of comparable software tooling performed to answer this question. Finally we will discuss the benefits and drawbacks of the tooling reviewed, evidencing the lack of an entirely suitable tooling for integration into the existing CS curriculum, thus motivating the need for novel software, which leads into the Methodology chapter.

2.1.1 Brief Introduction to Hoare Logic

Hoare Logic is a formal system for mathematically proving the correctness of software programs written in a simple imperative language, providing axioms and inference rules for common programming constructs. The central concept is the Hoare Triple, which formally represents the program state before (precondition) and after (postcondition) the execution of a program body (command), declaring the expected outcome on program properties (variable values). A program can consist of one or more Hoare Triples, and each triple's validity is determined by demonstrating its adherence to the relevant axioms and inference rules, often presented in a Fitch-style proof format, a common pedagogical approach as seen in textbooks like 'Logic in Computer Science' by Huth and Ryan (2004). An example involves a program where adding two to a variable X , initially 1, should result in X equaling 3.

$$\{X = 1\} X := X + 2 \{X = 3\}$$

Here ' $X=1$ ' is the precondition, ' $X:=X+2$ ' is the command ' $X=3$ ' is the post-

condition.

The proof would look as follows:

$X = 1 \rightarrow 1 + 2 = 3$ – Evidently true by arithmetic

$X = 1 \rightarrow X + 2 = 3$ – Simplify the expression

$X = 1 \rightarrow (X = 3)[X := X + 2]$ – Substitute the expression

$\{X = 1\}X := X + 2\{X = 3\}$ – Apply Assignment Law

This is a valid proof of the triple as it adheres to the assignment axiom schema with the program in question being an assignment operation.

2.1.2 Industry and Teaching Applications of Hoare Logic

Hoare Logic is a fundamental element of formal methods, inspiring and informing various software verification tools like Isabelle/HOL [2] for safety-critical systems and SPARK [13] for control systems like avionics [1]. Static analysis tools such as ESC/Java2 [14] have also utilized its principles, even contributing to the verification of voting systems [14]. Beyond verification, Hoare Logic principles guide the design of critical software like device drivers and ATM software, aiding in early bug detection and ensuring intended system behavior, ultimately contributing to safety and mitigating potential fatalities.

The significance of Hoare Logic has spurred the development of numerous teaching tools, including Key-Hoare, HABA, and CoQ, which are increasingly and successfully integrated into formal logic curricula. The HABA tool has statistically demonstrated improved student performance in Hoare Logic exercises [15]. Similarly, the Carnap System [11], supporting various logics including dynamic logic [16], has been associated with highly positive course evaluations at institutions like Kansas State University and is used across over 40 institutions, suggesting the broad pedagogical value of such tools in formal methods education. The industrial relevance of Hoare Logic underscores the importance of effective teaching, which, as evidenced by the success of tools like HABA and Carnap, can be significantly enhanced through dedicated software.

2.1.3 Context of Teaching Environment and Resulting Research Question

Theory-3 (THE3) is a mandatory second-year Computer Science module at the University of York that introduces formal methods, progressing from predicate logic to formal proof languages like natural deduction and Hoare Logic. The module is delivered in person through weekly lectures, problem classes (pen and paper exercises), and practical sessions (using software like Carnap or Isabelle). Carnap is used in the latter part of the module for teaching predicate and propositional logic[10], and while its impact at the university hasn't been formally studied, its long-term use in other institutions suggests educational value. Currently, Hoare Logic instruction in THE3 lacks dedicated practical sessions, consisting only of a problem class, although the module leader intends to increase Hoare Logic content, potentially including two practical sessions in the future. This lack of current tooling for Hoare Logic within THE3 provides the motivation for the research question: "Is there a suitable teaching tool for Hoare Logic for the delivery of THE3?".

2.2 Systematic Review of Existing Tooling

2.2.1 Introduction

To address the research question, a systematic review was conducted using YorSearch and Google Scholar with the initial search term 'teaching assistant hoare logic'. This yielded limited results, prompting an expansion to 'teaching assistant formal verification', which provided more general but relevant tooling examples. A further search for 'teaching assistant program verification' uncovered more industry-specific tools of relevance. Complementing this, online documentation for relevant tooling not found in academic archives was also consulted. The findings of this comprehensive review are detailed in the subsequent sections.

2.2.2 Existing Tooling

The following tools discussed discovered through the systematic literature review either directly implement support of Hoare Logic or implement closely related logics for the purpose of teaching formal logic courses. This allowed for comparative analysis of the toolings in respect to the features

2 Literature Review

they offered end-users, which where studies existed, were supported by the pedagogical value these features demonstrated.

One of the most popular tools discovered was the Carnap System as described earlier. The tooling itself is flexible and extensible and at time of writing is under active development. It provides a range of widgets which can be used to be construct problems to be solved/proved within a range of formal logics, e.g. propositional logic, modal logic etc.. These widgets can be used to construct problem sheets for students to work through and solve, which in turn help familiarize themselves with a range of concepts within the area of formal reasoning. The mathematical visualizer of the proof and instant feedback provided by the tooling were of great value when completing problems within the tool during manual testing. Tooling providing instant feedback is argued in numerous sources to support pedagogical effectiveness, one of such being a systematic survey conducted by the University of Michigan [17] in which it is argued that instant feedback on the errors made within a proof help to refine the mental model of how the proof system works, much like typical programming environments do.

However, it did have some drawbacks in the way of requiring memorization of exact ASCII characters to enter to represent logical formulae during proof construction. It was also arguable, that the use of a Fitch notation style for proof construction isn't as intuitive as a more sequential format that tools such as Key-Hoare use[18]. That said the argument for a 'Fitch-style' notation is still arguable, as it adheres more naturally to the Hoare Logic proof style and hence be a better fit for moving on from pen and paper exercises which implement the proof style.

Another tool which implements Hoare Logic is the HABA tool as previously mentioned. The tool offered the major benefit of providing a program development like interface for constructing Hoare Logic proofs. This is specifically referenced as a design intention of the HABA tool, 'we require the syntax of the programs to be close to original Hoare logic with possible extensions, but in the manner easily digestible by students who are acquainted with programming languages such as Pascal or Java.' . This requirement of the HABA software is derived from the instructor need to teach Hoare Logic in a format suitable for undergraduate teaching, as is a requirement of this project. Additionally, the eclipse environment uses error markers to highlight syntax errors when writing up programs and uses similar syntax highlighting to show computed errors from testing the validity of the triple, i.e. red underline on invalid post-condition for triple. This aspect of the tool ties into the benefit of instant feedback which as discussed Carnap also offers.

One major downside to the HABA tool, as is common for many formal methods tooling, is the difficulty in setting up the tool. This was partially

due to the issues involved with setting up the correct Java Runtime Environment(JRE) and version of Microsoft Z3 Simple Modulo Transformer(SMT) solver. Such issues with configuration can be a blocker to students, especially those on undergraduate courses, who in using difficult to configure software to understand such a tricky subject area may feel less incentivised towards developing their understanding, in turn being less likely to reap the rewards of using such tools. This is an issue faced not only by HABA but also tools such as Why3 and Logika in different regards as not only the dependencies but the software needing to be used in order to utilize them can vary widely. This makes many tools unsuitable at an introductory level, such as in an undergraduate teaching context. Another proof assistant is CoQ, a 'formal proof management system'. It provides a 'formal language to write mathematical definitions, executable algorithms and theorems together with an environment for semi-interactive development of machine-checked proofs.' It provides support for keybindings to access functionality in the user interface, such as settings and interestingly within jsCoQ provides a command-line focused approach to writing proofs. However, a downside of jsCoQ, similarly to the Carnap System, is how easy it is to forget essential syntax, due largely to both tools requiring you to learn a new Domain Specific Language(DSL). Which, while not always too impractical and a necessity within proof assistants to have a format to align proofs to, it was common to find these DSLs were too complex to easily engage with.

2.2.3 Tooling Review Outcome

It became clear from this investigation that while a myriad of tools exist, there was no one tool which would suffice for the teaching of Hoare Logic within the Theory-3 module. Therefore, it was decided to create a novel software to meet this need. However, in order to produce a suitable specification for such software, we needed further data to elucidate the requirements. This process is detailed within the methodology section.

3 Methodology

In this chapter, I cover the methodology utilized when carrying out this project. This covers the requirements gathering process, survey analysis, and intended proof form for the prototype to implement. Following this, the consideration of the different design approaches and determination of the software architecture are detailed, followed by the interface design to be followed and a brief overview of how the developed prototype will be evaluated.

The application was developed using the waterfall development methodology. This involved capturing and deriving the software requirements before development commenced, following the rigor of the former stage needing completing before the latter, unlike an agile approach which permits navigation between two sequential stages in adapting to changing requirements. This strategy was appropriate due to the combination of time limit restrictions as an Undergraduate project alongside the limited access to end-users for testing during prototype development. Thereby, making the possibility of implementation revisions based on evolving requirements impractical. There are five distinct stages in the waterfall approach, these are: requirements, design, implementation, testing, and deployment and maintenance. The final stage, deployment and maintenance, was not included in the project due to the created software being a proof of concept. The following process was constructed and followed to this methodology within the project.

3.1 Elicitation of Requirements

The requirements elicitation for a novel software prototype involved the synthesis of several sources of information. These being the user survey, gathered stakeholder requirements and my own review notes from my experiences using tooling during the systematic literature review. The manner and rationale for how these requirements were synthesized is detailed in this section.

3.1.1 User Survey

The user survey aimed to understand participants' comprehension of Hoare Logic and their experiences with related software tooling, gathering both positive and negative feedback to inform the design of a new tool. The survey specifically targeted areas where students struggled with Hoare Logic and directly asked for desired and undesired features for the prototype. It also sought volunteers for later prototype testing. The primary demographic was undergraduate Computer Science students at the University of York who had taken the Theory-3 module and used the Carnap System. The second demographic was PhD students in Formal Verification with Graduate Teaching Assistant(GTA) experience in the Theory-3 module. These responses aimed to capture the needs of the intended user base by identifying desired and undesired features and challenging theoretical areas. The undergraduate cohort's experience with Carnap was considered particularly relevant due to its comparable intended use case. While PhD students' advanced knowledge might seem less relevant, their GTA experience was expected to provide insights into effective teaching approaches. Overall, twelve responses were received, the analysis of which is discussed later, following a brief establishment of the value of other data sources.

3.1.2 Stakeholder Requirements

As for the stakeholder requirements, these refer to the criteria gathered from a small number of staff within the University of York's Computer Science department involved in the delivery of the Theory-3 Module, namely this project's academic supervisor, Simon David Foster, and PhD student, Christian Pardillo Laurence. These were collected informally through project meetings in the early stages of the project. The reason for gathering these requirements is similar to the overall intention of the first user survey in aiding elicitation of requirements for the prototype's target audience. However these capture a different but equally important perspective by providing criteria that should fit the exact requirements of the teaching delivery of the Theory-3 module. Therefore, being of direct value to what needs considering in a prototype tool. The captured list was ultimately quite short, but nonetheless for brevity of this paper they have been captured in the appendices.

3.1.3 Review Notes

The review notes mentioned capture the User Experience(UX) aspects, but equally the issues I encountered when investigating relevant tooling while performing the systematic literature review. These notes function to provide more specific criteria for what a prototype tool could include, given there was evidence of support generally for such criteria. For example, if I had found tooling didn't provide syntax-level error checking which informed me why a constructed proof was invalid, I would note this down as a frustration in my review notes. Then, during the requirements elicitation process, after having captured all the data sources declared at the start of this chapter. I would then look to the user survey to see if there were any general comments supporting participants frustration with lack of error checking. Considering the review notes in this manner, provided some additional assurance following the inclusion of the sources mentioned previously of what should be warranted to be a requirement.

3.1.4 Analysis of Survey Results

According to the conducted survey (see C C), the majority of participants as expected were undergraduate computer science students, primarily in the second year of their degree, but with a notable portion, specifically 25% in their final year (see C). The majority of participants had mostly used Carnap tooling as expected, which lines up with the data just identified as Carnap is employed in the teaching of the Theory-3 module at the University of York [10]. Additionally, the majority of participants assigned themselves a high level of understanding of Hoare logic, with 66.6% of participants rating at 7 or above out of 10 (see C). This statistic is of value as it can be argued it indicates the findings from responses on the whole tend to be based on solid understanding of Hoare Logic law application operates in proof construction and moreover that responses have an understanding of the difficulties involved in the learning process of Hoare Logic proof construction. Overall, although the specific manner in which this was indicated varied, responses showed that people appreciated and/or would appreciate instant feedback during use of tooling. For example, some individuals stated that they appreciated Carnap's ability to inform if the current proof was formatted correctly, i.e. 'In Carnap, the fact that it would report a problem with your logic as soon as you've written something that doesn't follow is very useful to prevent going down a floored path is useful'. Following this, other people appreciated tooling indicating when a proof has been successfully completed, i.e. 'The way the background of the question box lights up green when you get a question right is nice...', 'Immediate feedback on mistakes, as nice of a reward system as is possible (something

like a big green tick when you get a problem correct...'

In addition to the expressed interest in instant feedback, a related theme was interest in proof progress status. More specifically, people were interested in a tool elucidating why/why not a proof was correct or incorrect and how far along the construction of their proof was. While in theory these sound like they could be useful aspects of tooling, such features would need to be implemented with care as they could detract from students developing an understanding of the theory. A case of this may be if a developed tool allowed for the input of a Hoare Triple for which an end-user managed to prove the validity of by constructing a proof by a trial and error approach. This point is argued more generally in the paper 'Teaching Formal Methods: A Systematic Survey' [10] in regard to formal methods tooling such as Why3 and Dafny where it possibly allows students to take a 'trial-and-error' approach in constructing imperative program proofs due to the 'high level of automation' they provide. Thus, arguably if the prototype tool explained at the end of the proof why a proof is valid, having been able to take advantage of such automation facilities, without the end-user understanding the logic behind the proof construction themselves, i.e. the relevance of the proof axioms, the tool would have not been an aid in understanding the underlying theory. Moreover, aside from the potential for detriment to user understanding of the theory, similar requests bring additional complexity to tooling development, which on consideration of the time restraints involved with the project are therefore not justifiable to bring in to the application requirements. As a result of the above, the derived requirements from the survey, alongside the other sources previously detailed were paired down to ensure requirements would produce a prototype which is functionally simple but nonetheless an effective teaching aid.

3.1.5 Approach to Reconcile Collected Data

The requirements gathered from the user survey took precedence in importance in the sources stated above. The stakeholder requirements gathered supported the findings from the user survey, so a reconciliation process to handle conflicts was not needed. In the case of the review notes, as detailed previously these had least precedence in consideration for requirements ultimately derived due to their inherent bias, but however as also mentioned they did act to support a requirement being derived where relevant.

3.2 Resulting Software Requirements

Following the requirements elicitation process followed as discussed the Functional (FR) and Non-Functional (NFR) requirements were derived. The requirements were split into three categories, essential, desirable, and optional in both tables in order to align the NFR derived from each FR. The relevant NFRs were determined by typical standards for NFRs in the requirement generation process of web apps tailored by the time restrictions of the project, as a result the NFRs determined were arguably practical to achieve at the conception of prototype development.

Table 3.1: Functional Requirements Table

ID	Requirement
	Essential
1.01	Textual proof can be input into web app and determined to be correct, i.e. if proof shows whether Hoare Triple is valid or invalid
1.02	Proof requiring the 'hskip' law can be entered
1.03	Proof requiring the 'hassign' law can be entered
1.04	Proof requiring the 'hcond' law can be entered
1.05	Proof requiring the 'hseq' law can be entered
1.06	Proof requiring the 'hwhile' law can be entered
1.07	Proof written is displayed/formatted neatly within web app UI, i.e. required structure/format of proof is easy to parse
1.08	The web app clearly highlights syntax-related and rule mismatch errors within proof construction and displays the related error message
1.09	The web app user interface clearly indicates the successful completion of the proof
	Desirable
1.10	The web app offers the ability to save progress locally
1.11	The web app supports answering a range of Hoare Logic proof questions (from Theory-3 module teaching delivery)
1.12	The web app offers ability to create customisable problem sheets (for instructor use)
	Optional
1.13	The web app can display animation showing order of application of laws currently in current proof construction
1.14	The web app offers model answers of optimal proof solution for reference after completion of correct proof
1.15	The web app offers profile creation and progress dashboard facilities, i.e. tracking completion of instructor created/assigned problem sheets

3 Methodology

Table 3.2: Non-Functional Requirements Table

ID	Requirement
	Essential
2.01	The tool should be implemented as a web application
2.02	The web app should be designed primarily for use on wide format viewports e.g. desktop/laptop
2.03	The web app should adhere to the WCAG 2.1 guidelines
2.04	All web pages within web app must take less than 10 seconds to fully load all page content
2.05	The web app must perform without encountering failures 90% of the time it is operational
	Desirable
2.06	The tool must be scalable enough to support a maximum of 50 concurrent page loads while maintaining optimal load speed
2.07	The web application should be fully functional on all major web browsers, including Chrome, Firefox, Safari, and Edge, with consistent performance and layout.
2.08	The tool must have implemented maintainable/reliable provisioning (e.g. cloud, on-premises) of back-end infrastructure to store and deploy custom-created problem sheets, i.e. database, web-server
2.09	The back-end infrastructure of the tool should have relevant data considerations considered and handled, e.g. GDPR and data retention periods
	Optional
2.10	The tool must have implemented reliable provisioning (e.g. cloud, on-premises) of back-end infrastructure to handle CRUD operations for profile management and problem sheet completion progress
2.11	The communications and internal processing of sensitive data in the tool should be implemented while accounting for a reasonable level of security best practices, e.g. OWASP top 10
2.12	The tool's back-end infrastructure should be operational 90% of the time
2.13	The tool's backend relational database should be normalised to 3NF
2.14	The tool must handle CRUD operations successfully 90% of the time it is operational
2.15	The back-end infrastructure of the tool should have relevant data considerations considered and handled, e.g. GDPR and data retention periods

Following the creation of the requirements, the assessment of the technical approaches to create a tool to meet these requirements commenced. However, before we move onto this section we will detail and provide an example of the proof format for Hoare Logic proof construction decided upon.

This is detailed in the next section.

3.3 Intended Proof Form

The chosen format for constructing Hoare Logic proofs in the web-based tool is an adaptation of Fitch Notation [19]. This decision is motivated by Fitch Notation's widespread use and pedagogical value in Theoretical Computer Science (TCS) textbooks, exemplified by 'Logic in Computer Science' by Huth and Ryan, a standard text in many formal logic courses [20]. Furthermore, its familiarity to students through its use in the Carnap System within the Theory-3 Module is a key factor, aiming to ease the adoption of the new tool. The adapted format requires proofs to be structured in rows, following the backward logic approach inherent in Fitch Notation, where each line builds upon constructs to support the Hoare Logic law validating a triple. Unlike standard Fitch Notation, this adaptation omits assumptions and indentation for sub-proofs due to the rule-based proof construction not necessitating these features. Each proof line requires a suffix indicating the 'rule' applied, which can be either a Hoare Logic axiom or a construct from predicate logic and arithmetic used to justify the application of a Hoare Logic axiom, specifically for variable substitution, arithmetic operations, or simplification of arithmetic expressions.

The suffix used for each rule is detailed briefly below in table 3.3:

Table 3.3: Suffix for Proof Rule

Rule	Suffix
Hoare Logic Law	
Assignment Law	'hassign'
Skip Law	'hskip'
Conditional Law	'hcond'
Sequential Law	'hseq'
While Law	'hwhile'
Supporting Rule	
Variable Substitution	'subst'
Statement Simplification	'simpf'
Arithmetic	'arith'

Each line in a Hoare Logic proof, following a 'rule' suffix (either a Hoare Logic axiom or a predicate/arithmetic logic construct), can be further annotated with one or two digits referencing supporting proof lines. For instance, applying the Assignment Law requires a reference to the line performing variable substitution, which in turn refers to the line with the arithmetic operation, and that line refers to the simplification of the arithmetic. The simplification rule does not require a further reference, deemed a sufficient level of detail without needing predicate logic justifications. The final line for the Assignment Law clearly shows how the post-condition is met. In cases requiring triple decomposition (e.g., 'if-else' using the Conditional Law), each supporting triple's proof must be constructed independently, without mixing rules across different Hoare Logic axioms. Each supporting triple must be fully proven before the proof proceeds to the parent triple they support.

To elucidate what a proof of the form described would look like, the below example is given:


```

1   $x \leq y \rightarrow x = \min(x, y)$  : simpf
2   $x \leq y \rightarrow (z = \min(x, y))[z/x]$  : subst 1
3
4   $x > y \rightarrow y = \min(x, y)$  : simpf
5   $x > y \rightarrow (z = \min(x, y))[z/y]$  : subst 4
6
7   $\{x \leq y\}z := x\{z = \min(x, y)\}$  : hassign 2
8   $\{\neg(x \leq y)\}z := y\{z = \min(x, y)\}$  : hassign 5
9
10  $\{T\}\text{if } x \leq y \text{ then } z := x \text{ else } z := y\{z = \min(x, y)\}$  : hcond 7,8

```

3.4 Development Strategies

The primary objective of this project is to develop a user-friendly, web-based tool (as outlined in Functional Requirement 1.01) that enables students to construct and validate Hoare Logic proofs digitally. However, determining the validity of these proofs necessitates appropriate software tooling. While various tools exist, many are either impractical for integration or not specifically designed for the intended teaching context.

The selection of appropriate backend tooling involved considering Automated Theorem Provers (ATPs) like Vampire [21], which abstracts over proof construction (contradicting FR 1.01), and Interactive Theorem Provers (ITPs) such as Isabelle/HOL, whose desktop-based nature and licensing complexities made web adaptation impractical despite their support for step-by-step proofs (potentially addressing FR 1.09, 1.07). Ultimately, Satisfiability Modulo Theories (SMT) solvers, including Z3 [4] and cvc5 [22], emerged as the most flexible option for integration, often designed for embedding in larger systems (like Isabelle/HOL with Sledgehammer) via APIs, albeit requiring the development of a custom web interface. With these considerations in mind, the following development strategies on the interface were considered in order to come to an appropriate solution.

Option 1: Extend Open-Source Solutions

Numerous open-source software options exist with permissive licensing, including proof assistants like Carnap, HaHa, and CoQ, which function as ITPs. However, these often present complex interfaces unsuitable for undergraduate teaching and are not primarily web-based possessing complex code bases that would be impractical to adapt for the project's

specific Hoare Logic proof system and time constraints, as exemplified by jsCoQ and the client-side Haskell of Carnap . Despite these challenges, directly utilizing open-source SMT solvers like Z3 offers flexibility, although it also necessitates the development of a custom web interface.

Option 2: Start from scratch

This approach could be tricky as it would require having to implement not only the unification algorithm for Hoare Logic, but also a means to interpret and parse the VCs directly from the proof written by the user and then determine the validity of these using an implementation of Predicate Logic. This approach appeared to be the most time consuming of all the candidate options here, however it would provide the most flexibility as there would be no need to work around the complexities of large pre-developed systems, such as Carnap.

Option 3: Create novel solution utilizing existing software

This option is a compromise between options 1 and 2. The prototype could utilize an SMT solver which the details of the solver abstracted away with a web interface developed from scratch. SMT solvers can typically be ran externally to the web browser, such as within a deployment with a cloud provider such as AWS. Alternatively, the SMT solver could be attempted to be ran client-side directly on the clients web browser.

This approach would naturally bring some complexity but not as much as starting from scratch, as alongside utilizing the SMT solver, the process for creating the web app front-end could be made easier with use of modern front-end web frameworks and build systems, such as the web framework React.

3.5 Chosen Implementation Approach

Option 3 was chosen as the most appropriate option for this project. While option 1 offered the major benefits of being able to adapt a preexisting product which in some cases already provided the implementation needed for the Hoare Logic proof system. Unfortunately, the inherent complexity would make it impractical for the time constraints of this project to adapt the software to support the proof form. In the case of option 2, on a similar note the prospect of starting from scratch for the developed solution was anticipated to be too time consuming to implement. Therefore, the only practical option was Option 3, as it didn't require 'reinventing the wheel' in regard to determining fully if a Hoare Logic Law would be valid but also allowed the flexibility to interface with it using a modern web framework

that would help to ensure the interface speed, i.e. page loading time, was suitable for the prototype.

3.6 Software Decisions

TypeScript was chosen as the development language for the web application due to its type safety , which helps prevent runtime errors common in JavaScript , and its strong support by popular web frameworks like React and Angular , along with fast build systems such as Vite. For the SMT solver, Z3 , cvc5 , and Yices were considered. The final selection was based on factors like performance and platform support, specifically the feasibility of browser deployment versus requiring server side.

A detailed comparison of these solvers' pros and cons is provided in table 3.4 on the next page.

3 Methodology

Table 3.4: SMT Solver Benefits and Drawbacks

SMT Solver	Benefits	Drawbacks
Z3	• Used extensively in both academia and industry leading to significant testing and optimization over the years • Supports incremental solving, which can improve efficiency in interactive scenarios where proofs are built sequentially, such as in the case of this prototype • Has TypeScript bindings for use in Web Assembly(WASM) build, with the WASM Build being suitable for solving of smaller sized problems	• Client-side usage can be resource heavy due to size of Z3 WASM binary, increasing web app load times • Server-side implementation requires building and maintaining a backend service to handle VC generation and communication with Z3
cvc5	• CVC5 supports incremental solving, which can be beneficial, i.e. If a user modifies a proof slightly, the solver might be able to reuse some of its previous state, leading to faster re-verification. • High-performance SMT solver, often competing with or outperforming other leading solvers in various benchmarks and theories (as evidenced by SMT-COMP results).	• No fully fledged Web Assembly port of cvc5 or set of TypeScript bindings, meaning client-side implementation would be tricky to fully implement, server side implementation would be only practical approach
Yices	• Efficient as a solver for some theories, such as linear arithmetic where it outperforms Z3 and cvc5	• No existing Web Assembly port that could be developed upon or utilised directly, let alone supporting TypeScript bindings

Choice of SMT Solver:

After evaluating several SMT solvers (Z3, cvc5, and Yices) based on performance and existing documentation, Z3 was deemed the most appropriate choice. While its performance is comparable to cvc5, Z3's extensive academic and industrial usage ensures readily available documentation. Furthermore, its well-developed TypeScript bindings and WASM build, although potentially resource-intensive, offer acceptable performance for the smaller tasks typical in an educational setting. Despite the feasibility of both server-side and client-side deployment, the client-side option was selected for its ease of setup and to avoid potential configuration complexities associated with server-side deployment on cloud platforms like Amazon Web Services, streamlining the adoption process by keeping all application functionality self-contained within the web application.

3.7 Application Process Flow

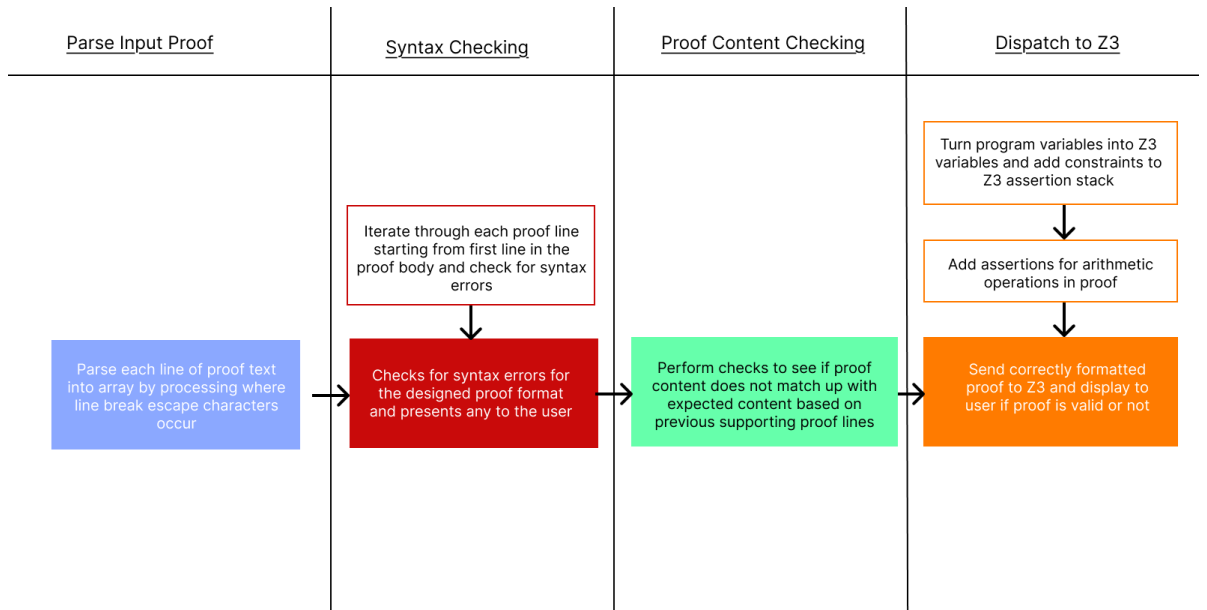


Figure 3.1: Application Process Diagram

Figure 3.1 shows the planned process flow that will be followed when a proof is submitted for validation in the prototype.

Each line in the proof is first formatted into an array for syntax level error checking. Syntax error checking, which refers to checking the proof line is

3 Methodology

constructed as expected for the 'rule' suffix used, e.g. a 'subst' rule suffix on a proof line would expect the string preceding the suffix to look like:

$$x=1 \rightarrow (x=3) [x > 3]$$

With the suffix alongside, the overall proof line would be of the following form:

$$x=1 \rightarrow (x=3) [x > 3] : \text{subst } 1$$

The frontend performs two main error checking stages. First, syntax errors are checked line by line, identifying issues like missing preconditions/post conditions or incorrect delimiters (e.g., '/' in multi-variable preconditions). Upon detecting a syntax error, a dialog box informs the user of the specific issue on the relevant line, aligning with user feedback requesting error checking (see C.1). If no syntax errors are found, the proof content is then checked for consistency based on predefined rules, ensuring that preconditions, post conditions, and program expressions correctly correspond across different proof lines (e.g., in substitution or arithmetic steps). Inconsistencies at this stage mark the proof as invalid without a specific error message, encouraging users to review their reasoning. Only after successfully passing both syntax and content checks is the proof dispatched to the Z3 solver for final validity determination, optimizing performance by avoiding unnecessary SMT solver calls for syntactically or semantically incorrect proofs. The technical details of this process are further elaborated in the Proof Kernel design section.

3.8 Design of Tooling Software

Having now come to a decision on the approach, I made some minor alterations to the requirements documents based on the practicality of the approach given anticipated development time constraints and contacted the stakeholders for their input. After receiving approval on the requirements, I mocked out a figma design of the interface of the Minimal Viable Product (MVP) of the prototype as a last step for review before commencing prototype development. The design produced is pictured in the figures below, additional figures can be found in the appendices (see appendix chapter D).

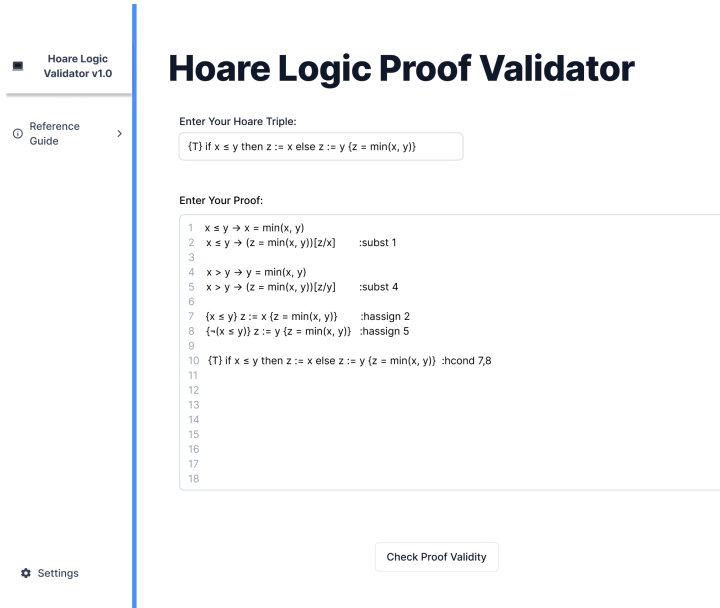


Figure 3.2: High-fidelity Design of Landing Page of Tool

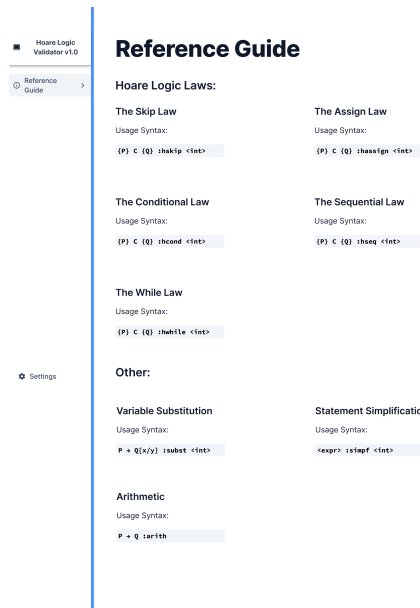


Figure 3.3: High-fidelity Design of Reference Guide Page

3.9 Brief Overview of Prototype Evaluation Criteria

In order to effectively determine how well the prototype meets the aim of the project, the success criteria had to be determined. The success criteria for this project refer simply to the functional requirements derived for this project, as in meeting these due to the evidence based process by which they were defined these would provide a direct approximation for the level of success the prototype attained in meeting the projects aims. In aid of evidencing the meeting of the success criteria, namely in showing the implementation of the proofs under the Hoare Logic laws, automated unit tests are used. Additionally, the adherence of the software to the digital accessibility requirements, namely the WCAG 2.1 guidelines are evidenced by the use of the Google Lighthouse tooling.

4 Implementation

In this chapter, I will cover the creation of the implemented software, starting with an overview of the software design, followed by a break down of then leading into the concept and explanation of the developed tool fulfilling its role as a proof kernel. Subsequently, the issues and limitations of the implemented product are covered followed by a summary of what was achieved within the implementation.

4.1 Overview

The software created was composed of two major parts, the first of these being the frontend web app interface and the second being the client-side JavaScript which parsed and processed the proof passed to the interface before passing to the client-side WASM module of the Z3 SMT solver through the JavaScript bindings. The tooling was version controlled using Git and iteratively deployed to Vercel as the web host and assigned a Uniform Resource Locator (URL) under the vercel domain. Deployments to the provider were initiated by a Github Actions(GA) workflow file when push or pull requests were made to the git branches named 'dev' and 'main'.

4.2 Design

4.2.1 Frontend Interface

The frontend web application was developed in TypeScript using the React framework 18.3.1 and the Vite build system v6.2. React's state management approach and Vite's fast prototyping with Hot Module Replacement (HMR) and performant production builds using Rollup with code-splitting and tree-shaking for optimized bundles made them suitable choices. The open-source frontend library shadcn/ui was used for its WCAG 2.1 compliance ensuring accessibility, its MIT license allowing free use and direct

component import for performance, and its utilization of Tailwind CSS for efficient stylesheet management .

4.2.2 Proof Kernel using Microsoft Z3

The Proof Kernel, a core component of the prototype , is inspired by N.G. de Bruijn's work on automated reasoning with the Automath project [23]. It's built using TypeScript files that process and convert textual proofs into a format digestible by the Z3 SMT solver , which is accessed via a WASM binary and JavaScript bindings provided by the 'z3-solver' NPM package. The kernel supports reasoning with up to three integer variables (domain 1-9, defined in preconditions using '=', '>', '<', '<=', '>=' and '/for multiple variables) and allows direct or proxy assignment. Program bodies support '*', '+', and '-'. Proof lines require a rule suffix and supporting line numbers (except the arithmetic rule, an exception that differed from the initial specification due to time constraints). Proofs are processed line by line, split by ", and each line undergoes syntax checking based on its rule suffix using Regular Expressions in a linear order (full details in appendix F.1, a snippet using the substitution rule checks is given as an example here).

Table 4.1: Syntax Error Checks For Substitution Rule

Rule	Applicable Checks
Substitution Rule	• Check correct substitution expression syntax, e.g. $[x >x+2]$ • Check precondition body content, e.g. $x=2$ • Check postcondition body content, e.g. $x=2$

In the case one of the above RegEx checks fail, a TypeScript object of the following form is returned:

```
{
errorDetails: String[]
isValid: Boolean
}
```

4 Implementation

'String[]' here refers simply to an array of strings which contain details on one syntax error while 'Boolean' simply refers to a boolean value of 'true' or 'false'. This return object is parsed into 'bubble dialog' errors and the border of the proof entry box is set to the colour red, as shown in the Figma design section.

Following syntax checks using regular expressions, proof content checks are performed, ensuring that referenced proof lines exist, are of the correct type (by comparing rule suffixes), and that preconditions, post conditions, and program expressions are present in the expected forms. Upon successful content checks, the appropriate function for interacting with the Z3 solver is invoked. Due to time limitations, only the Assignment Law and Skip Law were implemented, with the latter building upon the former to reduce complexity. The Skip Law function expects two arrays representing variable declarations before and after the implication, while the Assignment Law function requires one array for the content before the implication and an object containing an array of individual characters from the expression following the implication. The simplification rule was not implemented due to time constraints, making the arithmetic rule the foundational rule for subsequent law implementations.

Within Z3, each of the function calls creates a new 'scope' within the Z3 stack. Assertions and formulae, otherwise referred to as constraints, could be added to the Z3 stack and by creating a new scope, a space for assertions and formulae to be added that didn't get added to the Z3 stack was created. Although, realistically only one Hoare Logic proof was to be validated at a time, thus only one scope would ever need to exist at a time the choice was made as it was an effective means to keep a set of constraints for a satisfiability problem separated and making it easier to remove if need be, where it is more tricky to be sure you would be removing the correct constraint, therefore this approach benefits software extensibility.

For both Skip and Assignment Law proofs, the initial domain constraints for variables are directly translated into Z3 constraints. For Skip Law, the constraints from both the initial and subsequent proof lines are added to the Z3 stack, and the validity is checked by ensuring the implication of the initial constraints leading to the subsequent constraints is satisfiable, effectively verifying if the variable values remain consistent, although this isn't a full mathematical proof of the Skip Law. In contrast, for Assignment Law, while the initial domain constraints are handled similarly, the arithmetic part is validated by attempting to find a counterexample within a temporary Z3 scope; if no counterexample exists (unsatisfiable), the implication is considered valid and added to the main Z3 stack, otherwise, an error is flagged and propagated to the user interface, with the temporary scope then being cleared to maintain the Z3 stack's integrity.

4.3 Encountered Issues & Limitations

One major issue encountered during development was the delay encountered in loading the resources for the Z3 theorem prover into the browser context through the WASM module and JavaScript interface file. Although not an issue encountered on the local development server, when observing vercel deployments it was found that although the web interface loaded quickly proofs which pass the syntax and proof content error stages as outlined previously would result in the interface looking to be on 'stand-by' as the border colour of the proof entry box doesn't change. Or, perhaps arguably worse, the web app would timeout trying to elicit a response from the Z3 WASM module and falsely indicate the proof was invalid when it otherwise may not have been.

Another issue encountered was the inconsistency between where whitespace was allowed and not allowed in proof lines. On some proof lines, trailing whitespace would be permitted within the parts of a proof line, such as within the curly braces for a precondition, but when performing the 'proof content check' as previously described the RegEx implemented would mandate the use of no trailing whitespace resulting in a proof error to be displayed within the user interface unnecessarily. This is a hurdle to effectively delivering FR 1.08. This could be resolved by providing further development time on the software, however time constraints prevented this from being practical.

Additionally, it was encountered during development and subsequent manual testing of the software that the equality operators ' $\geq$ ' and ' $\leq$ ' when parsed in a proof did not return the expected result when checking the validity of assignment rule triples. Thus, it appears the logic created to handle the calling and construction of the relevant TypeScript bindings to appropriately determine Hoare Triple validity is insufficient, which while addressable would require additional development time which was impractical due to time constraints.

4.4 Implementation Summary

Overall, it seems much of the essential functionality was able to be implemented. In regard to the FRs, the prototype has a clean and responsive web UI, which allows for the entering of textual proofs. It highlight syntax-related and rule-mismatch errors and fundamentally the prototype is able to ascertain, admittedly only for a small sample of Hoare Logic Laws, but nonetheless if a Hoare triple is valid or not and clearly indicates this result

4 Implementation

to the user. As for the NFRs, the tool is implemented as a web application and certainly works on wide format viewports. Unfortunately, as mentioned support for a number of Hoare Logic laws was not able to be implemented, namely for the Conditional Law, Sequential Law and While Law. The major rationale why was purely due to time constraints, largely as the planned time to implement the prototype was insufficient for implementing the complexities of these laws on top of the base provided by the more fundamental laws, i.e. the Skip Law and Assignment Law. In addition on the NFR front, while the frontend appeared to meet the requirement of taking less than 10 seconds to fully load all visual content, the same cant be said for the Z3 dependencies themselves. However formal testing in the next chapter will determine this quantitatively.

There are numerous future technical improvements that could be made to the prototype, a simple one being checking of the Hoare Triple input in the Hoare Triple input box at the top of the page with the declaration of the triple in the overall proof body. The Hoare Triple input box was unfortunately unable to be integrated in this format due to time constraints, but if it were it would certainly be of aid in helping students ensure their proof construction follows the intended format in that the structure of their proof builds towards evidencing the Hoare Logic axioms relevance. Additionally and perhaps more crucial due to the hindrance to usability is the inconsistency of whitespace checking, detailed in the previous section as a known issue. Resolving this would be of great aid in better meeting FR 1.07.

Overall however the prototype does seem to meet a good deal of the essential functional and non-functional requirements, and where it doesn't it appears manageable. By this, I refer to the fact that not meeting the requirements for implementing some of the laws doesn't make the application not meet the overall aim of aiding in a teaching context. That said, the extent to which the essential requirements will be determined by testing.

5 Evaluation and Testing

The prototype developed was evaluated against the success criteria by a mixture of automated and manual testing. Following the collation of the results, the determination of the overall projects success was drawn.

5.1 Testing

Additionally for evaluating adherence to WCAG 2.1 guidelines and web page loading, the Google Lighthouse testing suite is employed on the most recent deployment of the tool on Vercel. The remaining FRs and NFRs in the success criteria not mentioned were somewhat subjective and due to lack of being able to distribute a survey to the original survey participants due to time restrictions, these requirements were evaluated by consideration of how the tool presented itself overall.

5.1.1 Automated Testing

Unit Testing

Unit tests were developed and executed to validate both the syntax checking and proof validity checking capabilities of the prototype, targeting the core functions responsible for these processes as detailed in the implementation chapter. These requirements are covered by functional requirements 1.01 - 1.03 and 1.08. These functions are directly invoked by the React web interface; managed by the Jest test runner, the results (shown in figure G.3) show successful proof validation of assignment and skip law proofs but unfortunately there are two failing syntax error tests. The test cases are derived from Theory-3 module problem sheets and custom examples and while not directly mimicking the process a user would navigate through, this method allows for a high fidelity evaluation of the core functionality of the prototype through being able to target specific functions used 'under the hood' despite not directly replicating user interaction. Further details on the tests are in appendices G.1 and G.2.

5 Evaluation and Testing

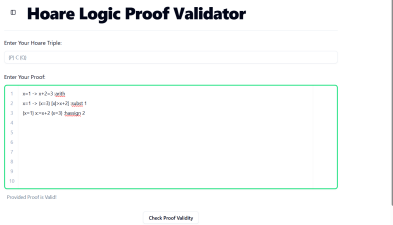
Valid Assignment Law Proof	Invalid Assignment Law Proof
Informal write-up: • $\{x = 1\} x := x + 2 \{x = 3\}$ • Apply hassign: $x = 1 \rightarrow (x = 3)[x \mapsto x + 2]$ • Substitute: $x = 1 \rightarrow x + 2 = 3$ • Simplify: $x = 1 \rightarrow 1 + 2 = 3$ • True by arithmetic. 	Informal write-up: • Apply hassign: $y > 1 \rightarrow (y > 2)[x \mapsto x + 1]$ • Substitute: $y > 1 \rightarrow y > 2$ • Clearly false, counterexample: $y = 2$. 

Table 5.1: Examples of Valid and Invalid Assignment Law Proofs

Web App Performance using Google Lighthouse

Google Lighthouse [5] is a tool used for measuring a websites performance, accessibility and Search Engine Optimisation(SEO). It was used to check the meeting of NFR 2.03 as it checks for adherence against WCAG 2.1 criteria in its automated testing suite. However, it was not used for NFR 2.04 as Google Lighthouse while able to check for page load times didn't account for public resources, namely the Z3 WASM module so it was felt a measure using this tool would be misleading thus a manual testing approach was opted for.

Testing was performed and Google Lighthouse indicated a clear adherence (see appendix H.2) to meeting WCAG 2.1 Guidelines, evidence of which can be seen in the appendices. Manual testing using the DevTools console in the Chrome browser were utilized to determine how long it took for the Z3 WASM binary to load in once page load was initiated. Tests were taken over spaced intervals of about 5 minutes each and the mean was calculated at 15.3 seconds (see table H.1), which while functional does unfortunately not meet the specification of NFR 2.04.

5 Evaluation and Testing

Table 5.2: Evaluation Against Success Criteria

Requirement ID	Requirement Summary	Rating	Comments
1.01	Textual proof can be input into web app and determined to be correct, i.e. if proof shows whether Hoare Triple is valid or invalid	2	Rated at 2 as not counting issues associated with implementation of specific rules which may affect only one rule type. This is based on measure of how well it actually reflects validity of proof, which due to error handling is for the most part fine but not perfect
1.02	Proof requiring the 'hskip' law can be entered	2	All unit tests pass hence meeting need for THE3 delivery of the rule
1.03	Proof requiring the 'hassign' law can be entered	2	All unit tests pass hence meeting need for THE3 delivery of the rule
1.04	Proof requiring the 'hcond' law can be entered	0	Not implemented
1.05	Proof requiring the 'hseq' law can be entered	0	Not implemented
1.06	Proof requiring the 'hwhile' law can be entered	0	Not implemented
1.07	Proof written is displayed/formatted neatly within web app UI, i.e. required structure/format of proof is easy to parse	2	User Interface is clear and easy to interact with for the most part, only downside is the inconsistent whitespace requirements in proof syntax
1.08	The web app clearly highlights syntax-related and rule mismatch errors within proof construction and displays the related error message	3	The tool supports a wide range of display of syntax errors on a line by line basis
1.09	The web app user interface clearly indicates the successful completion of the proof	3	Proof Entry box border always turns green or red on determining validity of proof, thus clear message to the user
2.01	The tool should be implemented as a web application	3	This requirement is evidently met
2.02	The web app should be designed primarily for use on wide format viewports e.g. desktop/laptop	3	Able to be used on both wide and mobile format viewports, as well as having passed testing WCAG 2.1 guidelines supporting this
2.03	The web app should adhere to the WCAG 2.1 guidelines	3	Evidenced by Google Lighthouse Audit
2.04	All web pages within web app must take less than 10 seconds to fully load all page content	2	Unfortunately not met as testing showed average load speed of Z3 WASM binary took 15.3 seconds

Key for Ratings: 0 = Not Met, 1 = Somewhat Met, 2 = Mostly Met, 3 = Completely Met

6 Conclusion

The software prototype produced ultimately meets the aim of this project, providing an easy to access proof kernel through the medium of a web application. The software performs syntax error highlighting by means of the instant feedback it provides while allowing for the construction of proofs using a format students are familiar with which is of direct aid in teaching Hoare Logic. The software also provides an easy to access reference sheet in the app, providing syntax reference for rule construction of proofs, a common user request from the requirements elicitation process (see Methodology chapter 3).

There are some drawbacks, such as issues with inconsistent whitespace requirements for formatting proofs and the delay involved in loading the Z3 WASM binary within a browser session and the occasional imperfect error handling surround this. Nonetheless however, the software created acts directly upon where previous tooling has lacked in meeting usability requirements for undergraduate teaching, thus pushing forward the state of the art in the delivery of effective teaching of the Hoare Logic proof system and to this extent the software has been successful.

A Appendices

B Link to Github Repository

<https://github.com/cyprste2717218/hoare-logic-proof-checker>

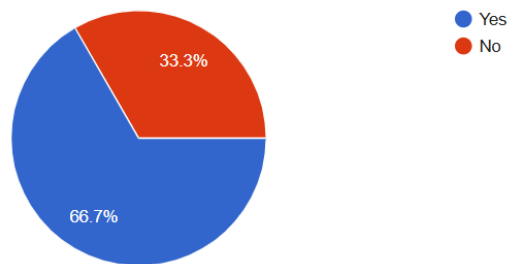
C Survey Data

It is planned that a prototype of the tool will be ready to test in Semester 2 (between S2/1 - S2/5).

 [Copy chart](#)

Would you be happy to participate in the testing of this tool followed by a brief survey on your experience?

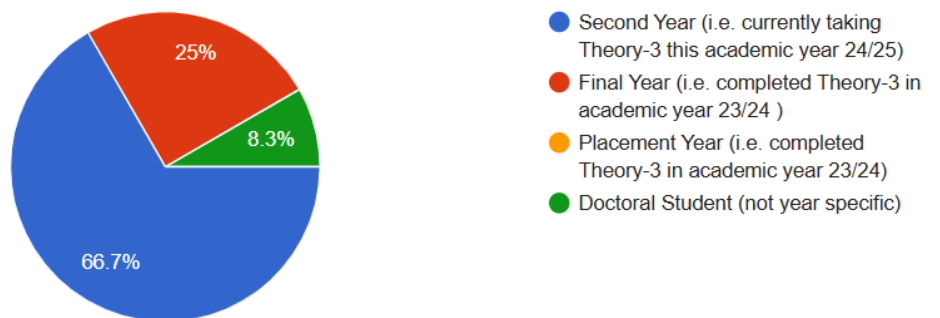
12 responses



What year group are you in?

 [Copy chart](#)

12 responses

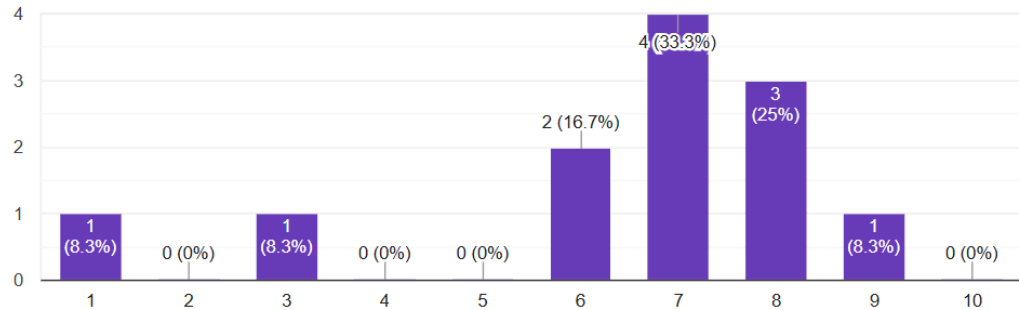


C Survey Data

On a scale of 1 - 10 with 1 being completely unfamiliar and 10 being very proficient,
How would you rate your understanding of Hoare logic?

[Copy chart](#)

12 responses



What course do you study? Alternatively if this does not apply, what is your area of research?

BSc Computer Science

MEng, Computer Science with Cyber Security Integrated Masters

Bachelors in CS

MEng Computer Science with Artificial Intelligence

Computer Science

Computer Science (with year in industry)

computer science

BEng Computer Science

Integrated Masters Mathematics and Computer Science (w/ Year in Industry)

Formal verification

Computer Science (MEng)

If you rated your understanding at 8 or above, were there any areas along your learning journey that you particularly struggled to understand?
N/A
No
Nothing major, but definitely remembering the formulae was an issue.
I found invariants difficult, I thought I understood them and then struggled to prove properties of seemingly simple programs which had me going back to the drawing board.
The idea of loop invariants for doing iterative statements. It took me a while to realise (and possibly I still don't fully understand) why you couldn't just choose any random invariant that is obviously always true (i.e. $x \neq x+1$)

What software tooling (or otherwise) have you used in the past to aid your understanding of formal/verification logic, including but not limited to program verification using Hoare logic?
Carnap
Carnap.
I have used Carnap.
CARNAP
Nothing - other than being shown Isabelle during the last lecture of the Semester
Isabelle/HOL
Carnap to do Natural Deduction proofs. I also use Rnote to sketch out my working digitally as if on paper

What did you like/dislike about these tool(s)? I.e. interface design, features available
I really like that it shows you where you've got to in terms of what you're trying to prove, and that it provides error hints. I like that there's a certain degree of flexibility in how you denote certain laws (e.g. being able to use '-' and '' to both mean 'not'). I dislike that the syntax availability is not consistent (for example, '!?' can be used to denote a contradiction, but '!?E' cannot be used as the 'bottom elimination' law), and also that the errors are sometimes quite unclear.
I liked being able to write statements using only my keyboard with no clicking and also the errors if something wasn't logical.
Carnap's immediate feedback was very useful - telling you if and where you went wrong, and when you had successfully completed a problem
It was hard to write the logical symbols and to get the right indentation.
dislike that there is nothing saying what each said rule translates to. like that it states by the side whether the rule you use is a verified rule
There were some bugs occasionally but it was fine overall
n/a
I really like the visual aid of showing the current proof with the indentation and lines (as Carnap does it) as it makes it really obvious what's going on, especially when case analyses or dealing with sub-proofs are involved. I don't like how you can't save your progress when partially answering a question and come back to it later.
Isabelle is a very powerful tool, and for experienced users it's great. An entirely text-based interface is flexible but inaccessible, and the language for building proofs could be better suited to Hoare logic.
In Carnap, the fact that it would report a problem with your logic as soon as you've written something that doesn't follow is very useful to prevent going down a floored path is useful. I find the tooltip where it tells you what format is expected when you get something wrong hard to understand.
The way the background of the question box lights up green when you get a question right is nice because its not over the top but very satisfying and makes it almost like you've won a game when you're trying to solve the problems.

(for GTAs only) In your role aiding with the teaching of the Theory-3 module, what have you noticed students struggle to understand when it comes to learning formal/verification logic, in particular Hoare logic. Do you feel any of these areas would/could be improved by software tooling?
N/A
I find that we haven't been able to teach students enough Hoare logic for it to be useful. We touch on invariants briefly, but the pace slows compared to the section where we utilise Carnap due to the lack of tool support. Students grasp the assignment rule fine, although some don't understand substitution immediately. The other laws prove a bit more difficult. There is also a switch from the rest of the module in terms of proof order - logic works fine with forward proof, where we start with the premises and finish at the conclusion, but Hoare logic is better suited for backwards proof. All of these areas would benefit greatly from software tooling.

The intention of this project is to build a web app to aid the teaching process of Hoare Logic, considering your previous responses are there any features would you want/not want in such a tool?
Basically those I like about Carnap (as stated above), and the inverse of those I'm not such a fan of about Carnap!
I would want to be able to use just my keyboard and also error checking/verification
Immediate feedback on mistakes, as nice of a reward system as is possible (something like a big green tick when you get a problem correct), quick access to the standard rules, model answers if you answer a problem less efficiently than possible / not how the question was intended to be solved
It would be nice to have a better input system for the logical symbols.
interactive questions that start of fairly basic while (if you include this) are explaining any concepts and then get harder.
ability to see the rules for Hoare logic ahead
Access to a syntax reference list and also a list of the logic rules
I would want the ability to view examples of Hoare Logic questions that gave line-by-line reasoning of why/what was done to progress to the next line. Perhaps completing each example, could trigger a statistic on your profile etc?? I would also want the ability to complete practice Hoare Logic questions and when I go wrong be prompted as to what I did wrong i.e. used the wrong law or proved it rather than disproving it???
Some kind of feature where you can get visualisations of the Hoare logic for each question would be super helpful. Also, having some kind of checking/verification of each line of what you have written would be super helpful.
I do not know what Hoare Logic is
Mainly a natural way to do backwards proof.
[immediate?] Feedback about when you get a question correct. Having the tool point out exactly where you have gone wrong will make it much easier to correct mistakes. Not sure where it would hinder the educational value but if there was kind of linting/auto-complete and syntax highlighting (e.g. You start typing 'if' and it would place the word 'then' in a lighter colour which could be accessed by pressing tab). Also making keywords bold / a different colour. It would be nice if your progress was saved, even if just locally within a single browser.

Are there any other thoughts/comments you would like to leave on this project?
Sounds like a great idea!
All the best :)
good luck
n/a
I was not able to select that I have not completed Theory 3 in one of the above questions, so I thought I should let you know that.
Consider dynamic logic for a program verification framework that could perhaps be easier to implement.
Good choice to make it a web application as it will have good cross-platform compatibility. I (as do many other CS Students) use Firefox as my primary browser so ensure that it works as intended on Firefox as well as Chrome.

D Additional Figma Design Images

D Additional Figma Design Images

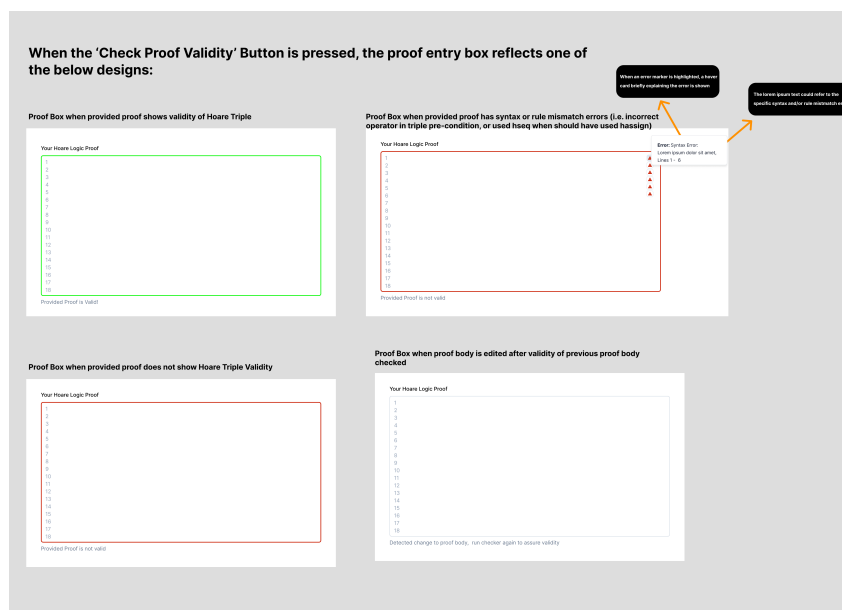


Figure D.1: After Pressing 'Check Proof Validity' Button

E Additional Prototype Images

E Additional Prototype Images

The screenshot shows the landing page of the Hoare Logic Proof Validator. On the left is a sidebar with the title 'Hoare Logic Validator v1.0' and three menu items: 'Options', 'Reference Guide' (which is active and has a right-pointing arrow), and 'Settings' (which also has a right-pointing arrow). The main content area has a header 'Hoare Logic Proof Validator' with a hamburger menu icon. Below the header, there is a section 'Enter Your Hoare Triple:' with a text input field containing the triple $\{P\} C \{Q\}$. Underneath is a section 'Enter Your Proof:' with a list of 10 numbered lines (1-10) and a large text area for the proof. At the bottom right of the main area is a button labeled 'Check Proof Validity'.

Figure E.1: Landing Page (Maximised Sidebar)

This screenshot shows the 'Reference Guide' section of the application. The sidebar on the left is identical to the previous figure, with 'Reference Guide' selected. The main content area has a header 'Reference Guide' with a hamburger menu icon. Below the header, the text 'Hoare Logic Laws:' is followed by a grid of five law entries. Each entry consists of a title, the text 'Usage Syntax:', and a code snippet in a light blue box. The laws are: 'The Skip Law' with $\{P\} C \{Q\} : \text{hskip } \langle \text{int} \rangle$; 'The Assign Law' with $\{P\} C \{Q\} : \text{hassign } \langle \text{int} \rangle$; 'The Conditional Law' with $\{P\} C \{Q\} : \text{hcond } \langle \text{int} \rangle$; 'The Sequential Law' with $\{P\} C \{Q\} : \text{hseq } \langle \text{int} \rangle$; and 'The While Law' with $\{P\} C \{Q\} : \text{hwhile } \langle \text{int} \rangle$.

Figure E.2: Reference Guide (Top Half)

E Additional Prototype Images

Hoare Logic Validator
v1.0

Options

Reference Guide >

Settings >

Hoare Logic Proof Validator

Enter Your Hoare Triple:

Enter Your Proof:

1 x=1 -> x=1 :arith

2 {x=1} x=1 {x=1} :hskip 1

3

4

5

6

7

8

9

10

Provided Proof is valid!

Check Proof Validity

Figure E.3: Valid Proof

Hoare Logic Validator
v1.0

Options

Reference Guide >

Settings >

Hoare Logic Proof Validator

Enter Your Hoare Triple:

Enter Your Proof:

1 x=1 -> x=4 :arith

2 {x=1} x=1 {x=1} :hskip 1

3

4

5

6

7

8

9

10

Provided Proof is not valid

Check Proof Validity

Figure E.4: Invalid Proof

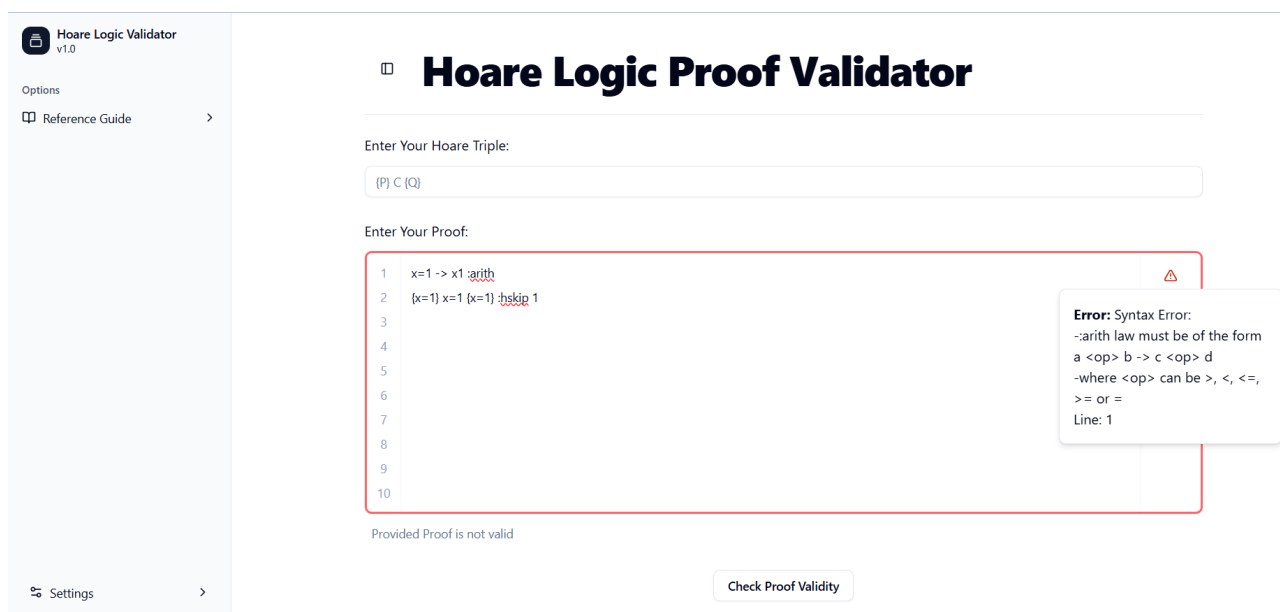


Figure E.5: Invalid Proof with Syntax Error

F All Syntax Error Rules Table

Table F.1: Syntax Error Checks For Rule

Rule	Applicable Checks	Error Message
Substitution	Check correct substitution expression syntax, e.g. $[x >x+2]$	'Substitution expression is incorrectly formatted, should be a single expression of the form $\langle \text{char} \rangle \langle \text{expr} \rangle$ enclosed between $[],$ e.g. $[x >x+2]$ '
Substitution	Check precondition body content, e.g. $x=2$	Precondition body is incorrectly formatted, should be singular/list of expressions of form $\langle \text{expr} \rangle \langle \text{operator} \rangle \langle \text{expr} \rangle$ delimited by $\wedge,$ e.g. $x=2 \wedge y=3$
Substitution	Check postcondition body content, e.g. $x=2$	'Postcondition body is incorrectly formatted, should be singular/list of expressions of form $\langle \text{expr} \rangle \langle \text{operator} \rangle \langle \text{expr} \rangle$ delimited by $\wedge,$ e.g. $x=2 \wedge y=3$ '
Hskip	Check precondition has open and closing braces	"Precondition does not contain both closing and opening braces, '{}'"
Hskip	Check precondition body content	'Precondition body is incorrectly formatted, should be singular/list of expressions of form $\langle \text{expr} \rangle \langle \text{operator} \rangle \langle \text{expr} \rangle$ delimited by $\wedge,$ e.g. $x=2 \wedge y=3$ '
Hskip	Check program body for a hskip proof line	'Program supplied to triple in Skip Law call is incorrectly formatted, should be a single expression or a list of expressions of the form $\langle \text{expr} \rangle \langle \text{operator} \rangle \langle \text{expr} \rangle$ delimited by $\wedge,$ e.g. $x:=3$ '
Hskip	Check postcondition has open and closing braces	"Postcondition does not contain both closing and opening braces, '{}'"
Hskip	Check postcondition body content	'Postcondition body is incorrectly formatted, should be singular/list of expressions of form $\langle \text{expr} \rangle \langle \text{operator} \rangle \langle \text{expr} \rangle$ delimited by $\wedge,$ e.g. $x=2 \wedge y=3$ '
Hassign	Check precondition has open and closing braces	"Precondition does not contain both closing and opening braces, '{}'"
Hassign	Check precondition body content	'Precondition body is incorrectly formatted, should be singular/list of expressions of form $\langle \text{expr} \rangle \langle \text{operator} \rangle \langle \text{expr} \rangle$ delimited by $\wedge,$ e.g. $x=2 \wedge y=3$ '
Hassign	Check program body for a hassign proof line	"Program supplied to triple in Assignment Law call is incorrectly formatted, should be a single expression of the form $\langle \text{char} \rangle := \langle \text{expr} \rangle,$ e.g. $x:=x+2$ "
Hassign	Check postcondition has open and closing braces	"Postcondition does not contain both closing and opening braces, '{}'"
Hassign	Check postcondition body content	'Postcondition body is incorrectly formatted, should be singular/list of expressions of form $\langle \text{expr} \rangle \langle \text{operator} \rangle \langle \text{expr} \rangle$ delimited by $\wedge,$ e.g. $x=2 \wedge y=3$ '
Arithmetic	Check content is delimited by arrow($\rightarrow$), e.g. of the form $x>1 \wedge y>1 \rightarrow x+y>2$	"'arith law must be of the form:', ' $a \langle \text{cop} \rangle b \rightarrow c \langle \text{mop} \rangle d \langle \text{cop} \rangle e$ ', 'where $\langle \text{cop} \rangle$ can be $>, <, \leq, \geq$ or $=$ ', 'and $\langle \text{mop} \rangle$ can be $+, -, /$ or $*$ ', 'e.g. $x=1 \rightarrow x+2=3$,'"

G Unit Test Results

G Unit Test Results

Passed: 4 | Failed: 0 | Pending: 0

Describe Block	Test Name	Status	Duration	Fail
Skip Law Valid Proofs	should validate single expression hskip proof	passed	1089ms	
Skip Law Valid Proofs	should validate multiple expression (length two) hskip proof	passed	178ms	
Skip Law Valid Proofs	should validate multiple expression (length three) hskip proof	passed	114ms	
Skip Law Valid Proofs	should validate expressions using <= and >= operators in hskip proof	passed	133ms	

Figure G.1: Skip Law Valid Proof

Passed: 4 | Failed: 0 | Pending: 0

Describe Block	Test Name	Status	Duration	Fail
Skip Law Valid Proofs	should validate single expression hskip proof	passed	1089ms	
Skip Law Valid Proofs	should validate multiple expression (length two) hskip proof	passed	178ms	
Skip Law Valid Proofs	should validate multiple expression (length three) hskip proof	passed	114ms	
Skip Law Valid Proofs	should validate expressions using <= and >= operators in hskip proof	passed	133ms	

Figure G.2: Skip Law Valid Proof

Test Suite Summary

 Passed: 27
 Failed: 5
 Pending: 0
Total Tests: 32
Pass Rate: 84.4%
Total Duration: 6.36s

Figure G.3: Summary of all Valid and Invalid Proofs

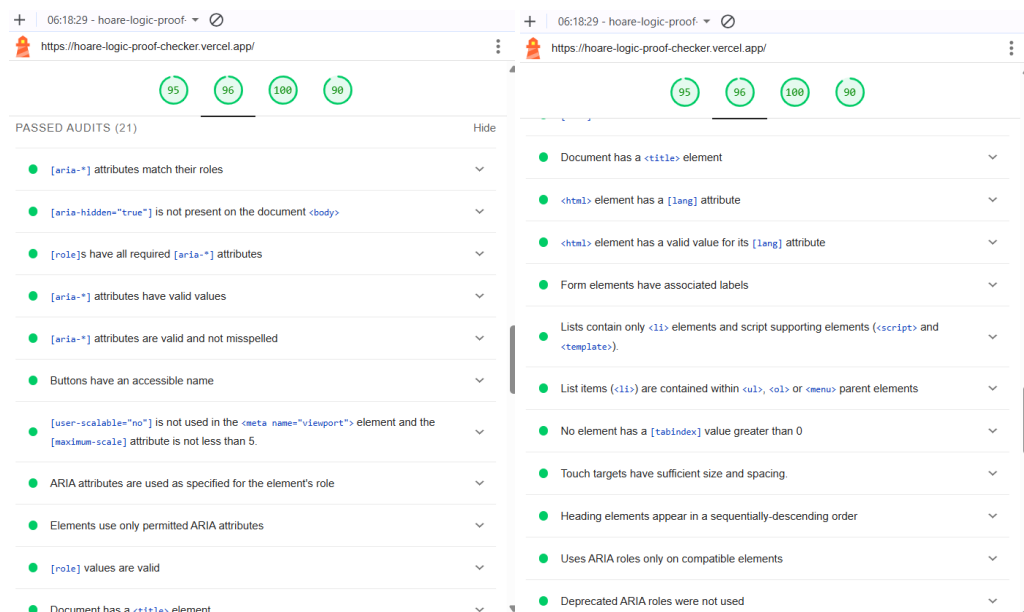
H Accessibility Test Results

H.1 Z3 WASM Load Time Tests

Trial 1	Trial 2	Trial 3	Mean
13	18	15	15.3

Table H.1: WASM module load time test (seconds)

H.2 Adherence to WCAG 2.1 Guidelines



Bibliography

- [1] R. Chapman, 'Formal Methods in Avionics: The Use of SPARK,' in *Proceedings of the 19th International Conference on Computer Safety, Reliability, and Security (SAFECOMP 2000)*, ser. Lecture Notes in Computer Science, vol. 1913, Springer, Berlin, Heidelberg, 2000, pp. 202–213. DOI: 10.1007/3-540-40849-8_17.
- [2] T. Nipkow, L. C. Paulson and M. Wenzel, *Isabelle/HOL - A Proof Assistant for Higher-Order Logic* (Lecture Notes in Computer Science). Springer, Berlin, Heidelberg, 2002, vol. 2283. DOI: 10.1007/3-540-45949-9.
- [3] W. W. Royce, '*Managing the development of large software systems: Concepts and techniques*, Proceedings of the 9th international conference on Software Engineering, blog.jbrains.ca, 1987, pp. 328–338., 1987.
- [4] L. de Moura and N. Bjørner, 'Z3: An Efficient SMT Solver,' in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, ser. Lecture Notes in Computer Science, vol. 4963, Springer, Berlin, Heidelberg, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.
- [5] Google, *Lighthouse - open-source, automated tool for improving the quality of web pages*, <https://developers.google.com/speed/docs/lighthouse>, Accessed: 11th June 2025.
- [6] C. A. R. Hoare, 'An axiomatic basis for computer programming,' *Communications of the ACM*, vol. 12, no. 10, pp. 576–583, 1969. DOI: 10.1145/363235.363259. [Online]. Available: <http://sunnyday.mit.edu/16.355/Hoare-CACM-69.pdf>.
- [7] R. Arthan, U. Martin and P. Oliva, 'A hoare logic for linear systems,' *Formal Aspects of Computing*, vol. 25, no. 3, pp. 345–363, 2013. DOI: 10.1007/s00165-011-0180-9.
- [8] K. R. M. Leino, E. M. Clarke and A. Voronkov, 'Dafny: An automatic program verifier for functional correctness,' in *Logic for Programming, Artificial Intelligence, and Reasoning*, A. Ben-Eliyahu-Zohary, N. Dershowitz and J. Palsberg, Eds., ser. Lecture Notes in Computer Science, vol. 6355, Germany: Springer Berlin / Heidelberg, 2010, pp. 348–370. DOI: 10.1007/978-3-642-17511-4_20.

Bibliography

- [9] C. Marché and Y. Moy, 'The jessie plugin for deductive verification in frama-c,' in *Abstracts of the 8th International Workshop on Satisfiability Modulo Theories (SMT 2012)*, 2012. [Online]. Available: <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.304.4108&rep=rep1&type=pdf>.
- [10] S. D. Foster, *Teaching Formal Methods: A Systematic Survey*. 2023.
- [11] G. Leach-Krouse, *Carnap: An Open Framework for Formal Reasoning in the Browser*. SYDNEY: Open Publ Assoc, 2018.
- [12] R. Hähnle and W. Mostowski, 'Key-hoare: A tool for verification of java programs using hoare logic,' in *Theorem Proving in Higher Order Logics*, ser. Lecture Notes in Computer Science, vol. 3603, 2005, pp. 276–292. DOI: 10.1007/11541868_19.
- [13] J. Barnes, 'SPARK: A Subset of Ada for Verified Software,' *High Integrity Software*, vol. 1, no. 4, pp. 239–248, 2003. DOI: 10.1002/hic.20008.
- [14] D. R. Cok and J. R. Kiniry, 'ESC/Java2: Uniting ESC/Java and JML,' in *Proceedings of the 18th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2004)*, ser. Lecture Notes in Computer Science, vol. 2988, Springer, Berlin, Heidelberg, 2004, pp. 108–122. DOI: 10.1007/b7613.10.
- [15] T. Sznuk and A. Schubert, 'Tool support for teaching hoare logic,' in *Software Engineering and Formal Methods*, D. Giannakopoulou and G. Salaün, Eds., ser. Lecture Notes in Computer Science, vol. 8702, Springer, Cham, 2014, pp. 369–383. DOI: 10.1007/978-3-319-10431-7_27. [Online]. Available: https://doi.org/10.1007/978-3-319-10431-7_27.
- [16] J. Tiuryn, 'Hoare logic: From first-order to propositional formalism,' in *Proof and System-Reliability*, ser. NATO Science Series, H. Schwichtenberg and R. Steinbrüggen, Eds., vol. 62, Springer, Dordrecht, 2002, pp. 165–183. DOI: 10.1007/978-94-010-0413-8_10. [Online]. Available: https://doi.org/10.1007/978-94-010-0413-8_10.
- [17] M. KEENAN and C. OMAR, 'Learner-centered design criteria for classroom proof assistants,'
- [18] R. Hähnle and R. Bubel, 'A hoare-style calculus with explicit state updates,' in *Formal Techniques for Java-like Programs*, ser. Lecture Notes in Computer Science, vol. 4988, Springer, Berlin, Heidelberg, 2008, pp. 90–109. DOI: 10.1007/978-3-540-79133-5_7.
- [19] J. Barwise and J. Etchemendy, 'The language of first-order logic, including the program "tarski's world" 4th edition,' *Language and Computation*, pp. 357–360, 1999. DOI: 10.1017/CBO9780511801606.013.

Bibliography

- [20] M. Huth and M. Ryan, *Logic in Computer Science: Modelling and Reasoning about Systems*, 2nd. Cambridge University Press, 2004.
- [21] A. Voronkov, 'The anatomy of vampire: Implementing bottom-up procedures with code trees,' in *Proceedings of the 15th International Joint Conference on Artificial Intelligence - Volume 1*, 1995, pp. 1414–1420. DOI: 10.5591/978-93-5107-411-1-vol1-p1414.
- [22] H. Barbosa, A. Reynolds, G. Kremer *et al.*, 'cvc5: A Versatile and Industrial-Strength SMT Solver,' in *Automated Reasoning: 11th International Joint Conference on Automated Deduction (IJCAR 2022)*, ser. Lecture Notes in Computer Science, vol. 13385, Springer, Cham, 2022, pp. 15–35. DOI: 10.1007/978-3-031-10769-6_3.
- [23] N. G. de Bruijn, 'The mathematical language AUTOMATH, its usage and some of its extensions,' *Symposium on Automatic Demonstration*, pp. 29–61, 1970. DOI: 10.1007/BFb0057957.